

对比

虚拟网卡

先创建虚拟网卡:

```
1 # 创建一对 veth设备, 命名为 veth0 和 veth1
2 sudo ip link add veth0 type veth peer name
   veth1
3
4 # 配置IP地址 (可选, 用于辅助测试)
5 sudo ip addr add 192.168.1.1/24 dev veth0
6 sudo ip addr add 192.168.1.2/24 dev veth1
7
8 # 启动设备
9 sudo ip link set veth0 up
10 sudo ip link set veth1 up
11
12 # 查看设备状态及MAC地址
13 ip link show veth0
14 ip link show veth1
```

熟悉可用的参数:

```

1  ./<build_dir>/examples/dpdk-l2fwd [EAL options]
   -- -p PORTMASK
2
3                                     [-P]
4                                     [-q NQ]
   --[no-]mac-
   updating
5                                     [--portmap="
   (port, port)[,(port, port)]"]

```

选取一些参数运行程序:

```

1  sudo ./l2fwd -l 0-1 -n 4 --
   vdev=net_pcap0,iface=veth0 --
   vdev=net_pcap1,iface=veth1 -- -p 0x3 --no-mac-
   updating

```

怎么控制frame

简单测试连通性,和是否被加密了.

```

1  from scapy.all import sendpfast, Ether
2  pkt = Ether(dst="02:00:00:00:00:02")/b'A'*128
3  sendpfast([pkt]*1000, iface="veth0", pps=10000)
   # 每秒 10000 pkt

```

尝试在另一边监测收到的frame:

```

1  sudo tcpdump -i veth1 -e -n -xx          # -e 显示
   以太头, -xx 显示原始内容

```

以字符串的形式显示:

```
1 | sudo tcpdump -i veth1 -A
```

比如我们之前发送了一个frame,tcpdump veth1的接口查看:

```
1 | sudo tcpdump -i veth1 -A
2 | tcpdump: verbose output suppressed, use -
  | v[v]... for full protocol decode
3 | listening on veth1, link-type EN10MB
  | (Ethernet), snapshot length 262144 bytes
4 | 19:53:45.804127 Loopback, skipCount 17736
  | (invalid)
5 | HELLO-DPDK
```

到这里还没有加密等等操作,我们才解决了发包控制的问题,可以控制长度和数量并且不会死循环.

testpmd做测试

加入简单的加密逻辑,

先运行这个命令程序:

```
1 | sudo ./build/app/dpdk-testpmd -l 0-3 -n 4 -- -i
  | \
2 | --forward-mode=macswap \
3 | --port-topology=chained \
4 | --burst=32
```

单线程处理

我们先不关心具体的加解密的逻辑,先考虑实现一个lcore处理两个端口上的收发和加解密,先用位运算模拟一下.

先调整一下参数,让一个**lcore**可以处理多个**port**上的收发问题,并且进行加密和解密.

```
1 sudo ./build/multi_thread/l2fwd/l2fwd -l 0 -n 4  
  --vdev=net_pcap0,iface=veth0 --  
  vdev=net_pcap1,iface=veth1 -- -p 0x3 -q 2
```

只要再加上真实的加密逻辑,我们就可以认为这样的一个线程处理了两个端口上的收包,发包和加密,解密的过程.

两个lcore处理两个端口:

```
1 sudo ./build/multi_thread/l2fwd/l2fwd -l 0-1 -n  
  4 --vdev=net_pcap0,iface=veth0 --  
  vdev=net_pcap1,iface=veth1 -- -p 0x3
```

A.时间的计算是否合理,为什么这里用一个线程比两个线程有时候更快?

1.采取更多数量的包实验.

2.dpdk库里面有没有更好的工具?

B.好像只给第一个包进行了模拟加密,原因?

我用数据包私有内存来判断,它直接被free,但是数据残留,所以,这两个问题是一起解决的.都进行"加解密"

暂时利用发送后更改etherType,收到后验证etherType解决了这个问题,OK

具体加解密流程复杂,怎么实现?

C.单线程实现之后,怎么分离线程工作的逻辑,单独拿一部分来收发,另一部分加解密工作,命令行参数是怎样的?

现在还没有放入真实的加密逻辑,先对比一下一个线程处理两个端口和两个线程分别处理两个端口的情况.

1000个包作为基准,双lcore:

```
1 Forwarded 1000 frames in 19.51784 seconds
2 The speed is 51.23518 frame/s...
```

一个lcore:

```
1 Forwarded 1000 frames in 18.30551 seconds
2 The speed is 54.62837 frame/s...
```

并没有明显的性能差异.(这个时候实现有问题,而且速度太慢)

加解密的问题

接下来我们要搞清楚的是,单线程加解密,或者加解密本身是怎样的流程.

先使用简单的加解密的逻辑,看看有没有变慢:

接下来我们会拿5000个包做测试.

之前的payload是128字节,我们改成1024字节尝试一下:

py构造数据包是否有速度的限制?用tcpreplay来放包.--->是这个问题,感觉之前的对比没有什么意义了.

那么现在我们引入流水线来处理,收发包的逻辑和加解密的逻辑分开.

多线程流水线处理

那么我们之前的实现的代码就是单线程处理的.

怎么把I/O和加解密解耦?

要使用rte_ring来解决.

结果

我们进行的简单对比:

情况

- A. 一个程序仅使用一个lcore,处理两个端口的全部收发和加解密.
- B. 另一个使用两个lcore,使用rte_ring,其中一个lcore负责处理两个端口上面的收发以及信息的打印,一个lcore仅负责处理ring队列中的加解密.
- C. 不解耦,2个lcore分别处理两个端口,参与收发和加解密.

测试

用**dpdk-testpmd**来做产生流量进行测试.我们对于一个网口进行流量发送,然后查看效果.

```
1 sudo dpdk-testpmd -l 2-3 -n 4 \  
2     --vdev=net_pcap0,iface=veth0 \  
3     --file-prefix=testpmd \  
4     --proc-type=auto \  
5     -- --forward-mode=txonly \  
6     --txd=1024 --rx=1024 \  
7     --port-topology=loop \  
8     --nb-ports=1 \  
9     -i
```

然后进行单向发送的配置:

```
1 testpmd> stop
2 testpmd> port stop all
3 testpmd> port config all txq 1
4 testpmd> port config all rxq 1
5 testpmd> set fwd txonly
6 testpmd> set txpkts 64
7 testpmd> port start all
8 testpmd> show config fwd      # 确认配置
9 testpmd> start tx_first
```

这样的配置应该没有问题:

```
1 txonly packet forwarding - ports=1 - cores=1 -
  streams=1 - NUMA support enabled, MP allocation
  mode: native
2 Logical Core 3 (socket 0) forwards packets on 1
  streams:
3   RX P=0/Q=0 (socket 0) -> TX P=0/Q=0 (socket
  0) peer=02:00:00:00:00:00
```

以5000000个数据包做基准,虽然我不知道会不会有性能的限制.

128bytes:

A:

```
1 Forwarded 5000000 frames in 12.38187 seconds
2 The speed is 403816.27546 frame/s...
```


B:

```
1 Forwarded 5000000 frames in 7.66612 seconds
2 The speed is 652220.43639 frame/s...
```

C:

```
1 Forwarded 5000000 frames in 10.39675 seconds
2 The speed is 480919.31557 frame/s...
```

$B/A = 1.615$

$C/A = 1.190$

加速比:1.357

256bytes

A:

```
1 Forwarded 5000000 frames in 13.21581 seconds
2 The speed is 378334.88000 frame/s...
```

B:

```
1 Forwarded 5000000 frames in 8.38385 seconds
2 The speed is 596384.51196 frame/s...
```

C:

```
1 Forwarded 5000000 frames in 11.45924 seconds
2 The speed is 436329.14321 frame/s...
```

$B/A = 1.576$

$C/A = 1.1532$

加速比:1.37

512bytes

A:

```
1 Forwarded 5000000 frames in 15.33425 seconds
2 The speed is 326067.56745 frame/s...
```

B:

```
1 Forwarded 5000000 frames in 9.44720 seconds
2 The speed is 529257.27606 frame/s...
```

C:

```
1 Forwarded 5000000 frames in 11.91589 seconds
2 The speed is 419607.71766 frame/s...
```

$B/A = 1.625$

$C/A = 1.286$

加速比:1.26

1024bytes

A:

```
1 Forwarded 5000000 frames in 18.11797 seconds
2 The speed is 275969.12640 frame/s...
```

B:

```
1 Forwarded 5000000 frames in 12.01628 seconds
2 The speed is 416102.19014 frame/s...
```

C:

```
1 Forwarded 5000000 frames in 13.28649 seconds
2 The speed is 376322.06287 frame/s...
```

$$B/A = 1.5077$$

$$C/A = 1.3636$$

加速比:1.1056

结论

因为之前并没有做过性能测试相关的工作,所以肯定有很多不严谨和不对的地方,仅从实验结果的数据来看,加速比是随着frame大小的上升而降低的.

原因是什么?

猜测:I/O和加解密的工作肯定都会变多,但是I/O带来的开销更大,导致I/O和加解密解耦的工作方式的效率下降.

但是我这里的多线程并不明显,因为只有两个线程,一个来负责收发,一个负责加解密.